

R2.01 - Développement Orienté Objets

Implémenter des classes en Kotlin

Arnaud Lanoix Brauer

Arnaud.Lanoix@univ-nantes.fr



Nantes Université

Département informatique

Déclarer une classe en Kotlin

```
class Chien {  
    var nom : String = ""  
    var age : Int = 0           // en mois  
    var race : String = ""  
    var poids : Double = 0.0 // en kg  
  
    fun aboyer() {  
        println("$nom dit : ouaf !!!")  
    }  
  
    fun renommer(nouveauNom : String) {  
        nom = nouveauNom  
    }  
  
    // @param distance en m  
    fun courir(distance : Int) {  
        // le chien perd 1 g / km  
        poids -= (distance / 1000.0) / 1000  
    }  
  
    fun ageEnAnnee() = age / 12.0  
}
```

Chien
+nom : String = ""
+age : Int = 0
+race : String = ""
+poids : Double = 0.0
+aboyer()
+renommer(nouveauNom : String)
+courir(distance : Int)
+ageEnAnnee() : Double

- Une classe est déclarée grâce au mot-clef **class**
- Les **attributs** sont déclarés comme des variables **internes** à la classe
 - ▶ Les attributs doivent être initialisés
- Les **méthodes** sont déclarées comme des fonctions **internes** à la classe
 - ▶ On peut **consulter** ou **modifier** les valeurs des attributs via les méthodes

Utiliser une classe en Kotlin

```
fun main() {  
    var rogue = Chien()  
    rogue.nom = "Rogue"  
    rogue.age = 15  
    rogue.race = "Berger Australien"  
    rogue.poids = 30.2  
  
    var potter = Chien()  
    potter.nom = "Other"  
    potter.age = 40  
    potter.race = "Beauceron"  
    potter.poids = 39.4  
  
    rogue.aboyer()  
    potter.renommer("Potter")  
    println("${potter.nom}:  
        ${potter.poids} kg")  
    potter.courir(400_000)  
    println("${potter.nom}:  
        ${potter.poids} kg")  
    var age = rogue.ageEnAnnee()  
    println("${rogue.nom}: $age ans")  
}
```

- On **instancie** des variables de type `Chien`
- On utilise la notation `.` pour
 - ▶ **accéder** aux attributs d'un objet
 - ▶ **appeler** des méthodes sur un objet

Attention

ce n'est pas une bonne idée d'initialiser les attributs un par un, comme ici :

- risque d'oublier certains attributs
- problème en cas de nouvel attribut

Il faut passer par un **constructeur**

Utiliser une classe en Kotlin

```
fun main() {  
    var rogue = Chien()  
    rogue.nom = "Rogue"  
    rogue.age = 15  
    rogue.race = "Berger Australien"  
    rogue.poids = 30.2  
  
    var potter = Chien()  
    potter.nom = "Other"  
    potter.age = 40  
    potter.race = "Beauceron"  
    potter.poids = 39.4  
  
    rogue.aboyer()  
    potter.renommer("Potter")  
    println("${potter.nom}:  
        ${potter.poids} kg")  
    potter.courir(400_000)  
    println("${potter.nom}:  
        ${potter.poids} kg")  
    var age = rogue.ageEnAnnee()  
    println("${rogue.nom}: $age ans")  
}
```

- On **instancie** des variables de type `Chien`
- On utilise la notation `.` pour
 - ▶ **accéder** aux attributs d'un objet
 - ▶ **appeler** des méthodes sur un objet

Attention

ce n'est pas une bonne idée d'initialiser les attributs un par un, comme ici :

- risque d'oublier certains attributs
- problème en cas de nouvel attribut

Il faut passer par un **constructeur**

Constructeur en Kotlin

```
class Chien (monNom : String,
            race : String = "inconnue",
            poids : Double) {
    var nom : String
    var age : Int = 1           // en nb de mois
    val race : String
    var poids : Double         // en kg
    init {
        nom = monNom
        this.race = race
        this.poids = if (poids > 0.0) poids else 0.1
    }
}
```

Chien

```
+nom : String
+age : Int = 1
+race : String
+poids : Double
+«create» Chien(monNom : String, race : String = "inconnue", poids : Double)
```

- les **paramètres** du constructeur sont déclarés après le nom de la classe
- On peut définir des **valeurs par défaut** pour les paramètres
- tous les attributs ne sont pas forcément présents comme paramètres
- les paramètres **initialisent** directement les attributs ou dans la clause `init {...}`
- tous les attributs doivent être initialisés
- si un attribut et un paramètre portent le même nom, on distingue l'attribut grâce à `this.`

Constructeur en Kotlin : utilisation

```
var rogue = Chien("Rogue", "Berger Australien", 30.2)
var potter = Chien("Other", "Beauceron", 39.4)
var soukie = Chien("Soukie", 4.7) // race non precisee => valeur par default

rogue.aboyer()
potter.renommer("Potter")
soukie.aboyer()
```

Les objets sont **instanciés** par l'appel du constructeur, en **valant** les différents paramètres nécessaires

Attributs déclarés immuables (`val`)

Attributs `val`

Cela peut avoir beaucoup de sens de déclarer certains attributs **immuables** afin d'exprimer qu'une fois initialisés, ils ne peuvent plus être modifiés

```
class Chien(nom : String, age : Int = 1,
            race : String = "inconnue", poids : Double) {
    var nom : String
    var age : Int      // en nb de mois
    val race : String
    var poids : Double // en kg

    init{
        this.nom = nom
        this.age = age
        this.race = race
        this.poids = if (poids > 0.0) poids else 0.1
    }
}
```

Ici l'attribut `race`, pour un chien donné ne doit plus pouvoir être changé : cela voudrait dire qu'on change la race d'un chien, ce qui est impossible

Visibilités en Kotlin

Par défaut, la **visibilité** des attributs et des méthodes est `public`

Attributs et méthodes **peuvent être déclarés** `private` afin de **restreindre** leur accès direct

- Certains attributs doivent être (en partie) cachés pour **contrôler** les modifications possibles de la classe : **principes d'abstraction et d'encapsulation**
- On peut être plus fin en interdisant uniquement l'accès **en écriture**
 - ▶ en UML : attribut `public` + propriété `{readOnly}`
 - ▶ en Kotlin : `private set` interdit uniquement la modification de l'attribut
- Certaines méthodes sont simplement **utilitaires** et ne doivent pas pouvoir être appelées depuis une autre classe

Attention

Les attributs `private` peuvent toujours être consultés/modifiés par l'intermédiaire de méthodes.

Visibilités en Kotlin

Par défaut, la **visibilité** des attributs et des méthodes est `public`

Attributs et méthodes **peuvent être déclarés** `private` afin de **restreindre** leur accès direct

- Certains attributs doivent être (en partie) cachés pour **contrôler** les modifications possibles de la classe : **principes d'abstraction et d'encapsulation**
- On peut être plus fin en interdisant uniquement l'accès **en écriture**
 - ▶ en UML : attribut **public** + propriété `{readOnly}`
 - ▶ en Kotlin : `private set` interdit uniquement la modification de l'attribut
- Certaines méthodes sont simplement **utilitaires** et ne doivent pas pouvoir être appelées depuis une autre classe

Attention

Les attributs `private` peuvent toujours être consultés/modifiés par l'intermédiaire de méthodes.

Visibilités en Kotlin

Par défaut, la **visibilité** des attributs et des méthodes est `public`

Attributs et méthodes **peuvent être déclarés** `private` afin de **restreindre** leur accès direct

- Certains attributs doivent être (en partie) cachés pour **contrôler** les modifications possibles de la classe : **principes d'abstraction et d'encapsulation**
- On peut être plus fin en interdisant uniquement l'accès **en écriture**
 - ▶ en UML : attribut **public** + propriété `{readOnly}`
 - ▶ en Kotlin : `private set` interdit uniquement la modification de l'attribut
- Certaines méthodes sont simplement **utilitaires** et ne doivent pas pouvoir être appelées depuis une autre classe

Attention

Les attributs `private` peuvent toujours être consultés/modifiés par l'intermédiaire de méthodes.

Visibilités en Kotlin : exemple

```
class Chien (...) {  
    var nom : String  
    private var age : Int  
    val race : String  
    var poids : Double  
        private set  
  
    fun courir(dist : Int) {  
        poids -= poidsEnMoins(dist)  
    }  
  
    private fun poidsEnMoins(d : Int)  
        = (d / 1000.0) / 1000  
}
```

Chien
+nom : String
-age : Int
+race : String
+poids : Double {readOnly}
+«create» Chien(...)
+courir(dist : Int)
-poidsEnMoins(d : Int)

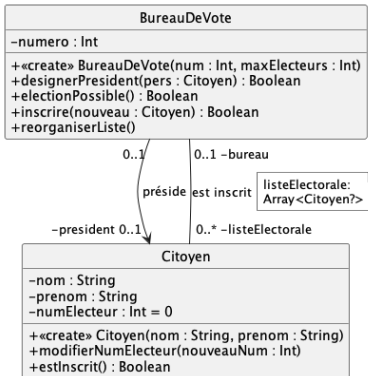
```
println("${rogue.nom}")  
rogue.nom = "Severus"  
println("${rogue.age}")           // error  
rogue.age = 10                    // error  
println("${rogue.race}")  
rogue.race = "Serpentard"         // error  
println("${rogue.poids}")  
rogue.poids = 42.0                // error  
rogue.courir(100)  
rogue.poidsEnMoins(100)           // error
```

- L'attribut `nom` est `public` (par défaut) : accessible en lecture/écriture
- L'attribut `age` est `private` : aucun accès possible
- L'attribut `race` est `public`, mais immuable : accessible en lecture
- L'attribut `poids` est restreint en écriture : accessible **uniquement** en lecture
- La fonction `courir()` est `public` (par défaut) : accessible
- La fonction `poidsEnMoins()` est `private` : aucun accès possible

Exemple Citoyen : implémentation en Kotlin

On a déjà précisé comment implémenter beaucoup d'éléments :

- types des attributs, méthodes, constructeurs, etc.



Comment implémenter les associations

- Une association **unidirectionnelle** devient un **attribut** dans la classe "source"
- Une association **bidirectionnelle** devient **deux attributs**, un de chaque côté de l'association,
- Les **rôles** deviennent les **noms** des attributs à ajouter
- Les cardinalités **influent** sur l'implémentation ,
- (si nécessaire, ajouter des méthodes pour **mettre à jour** les nouveaux attributs)

```

class Citoyen (nom : String,
               prenom : String) {

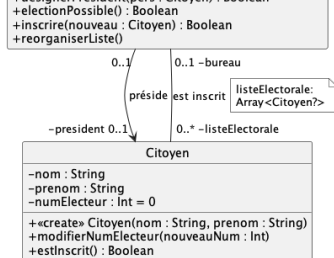
    private var nom : String
    private var prenom : String
    private var numElecteur : Int
    private var bureau : BureauDeVote?

    init {
        this.nom = nom
        this.prenom = prenom
        this.numElecteur = 0
        this.bureau = null
    }

    fun modifierNumElecteur(nouveauNum : Int) { /* todo */ }

    fun estInscrit() = false // todo
}

```



Cardinalité 0..1

L'association "est inscrit" à une cardinalité 0..1 coté "Bureau" : cela donne un attribut bureau de type Bureau?, càd Bureau possiblement **null**

```

class Citoyen (nom : String,
               prenom : String) {

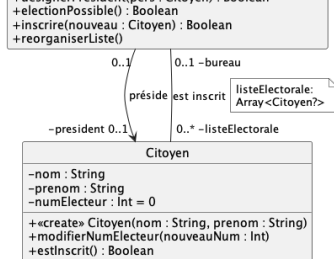
    private var nom : String
    private var prenom : String
    private var numElecteur : Int
    private var bureau : BureauDeVote?

    init {
        this.nom = nom
        this.prenom = prenom
        this.numElecteur = 0
        this.bureau = null
    }

    fun modifierNumElecteur(nouveauNum : Int) { /* todo */ }

    fun estInscrit() = false // todo
}

```



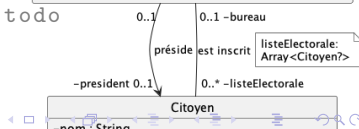
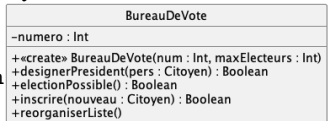
Cardinalité 0..1

L'association "est inscrit" à une cardinalité 0..1 coté "Bureau" : cela donne un attribut bureau de type Bureau?, càd Bureau possiblement **null**

Cardinalité 0..*

L'association "est inscrit" à une cardinalité 0..1* coté "Citoyen" : cela donne un attribut listeElectorale de type `Array<Citoyen?>`, càd un tableau de `Citoyen`

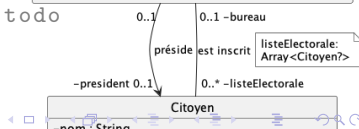
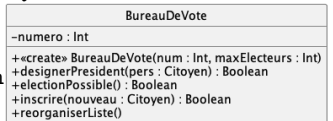
```
class BureauDeVote (num : Int, maxElecteurs : Int) {  
  
    private val numero : Int  
    private var president : Citoyen?  
    private val listeElectorale : Array<Citoyen?>  
  
    init {  
        numero = num  
        president = null  
        listeElectorale = arrayOfNulls<Citoyen>(maxElecteurs)  
    }  
  
    fun designerPresident(pers : Citoyen)  
  
    fun electionPossible() = false // todo  
  
    fun inscrire(nouveau : Citoyen)  
        = false // todo
```



Cardinalité 0..*

L'association "est inscrit" à une cardinalité 0..1* coté "Citoyen" : cela donne un attribut listeElectorale de type `Array<Citoyen?>`, càd un tableau de `Citoyen`

```
class BureauDeVote (num : Int, maxElecteurs : Int) {  
  
    private val numero : Int  
    private var president : Citoyen?  
    private val listeElectorale : Array<Citoyen?>  
  
    init {  
        numero = num  
        president = null  
        listeElectorale = arrayOfNulls<Citoyen>(maxElecteurs)  
    }  
  
    fun designerPresident(pers : Citoyen)  
  
    fun electionPossible() = false // todo  
  
    fun inscrire(nouveau : Citoyen)  
        = false // todo
```




```

fun designerPresident(pers : Citoyen) =
    if (pers.estInscrit()) {
        president = pers
        true
    }
    else false
//
private fun dernierePosition() : Int {
    for (i in 0 until listeElectorale.size)
        if (listeElectorale[i] == null) return i
    return listeElectorale.size
}
//
fun inscrire(nouveau : Citoyen) : Boolean {
    val position = dernierePosition()
    if (nouveau.estInscrit()           // nouveau deja inscrit
        || listeElectorale.size == position) // liste pleine
        return false
    listeElectorale.set(position, nouveau)
    nouveau.modifierBureauDeVote(this)
    nouveau.modifierNumElecteur(dernier+1)
    return true
}

```

Résumé UML → Kotlin

UML tous les attributs doivent être **typés**

- ▶ Toutes les **visibilités** doivent être précisées

UML il doit y avoir un **constructeur** : `<<create>>`

Kotlin les **associations** UML deviennent des **nouveaux attributs** Kotlin

- ▶ association **bidirectionnelle** : 2 attributs
- ▶ association **unidirectionnelle** : un attribut, uniquement dans la classe source
- ▶ les **rôles** nomment ce(s) nouvel(-aux) attribut(s)

Kotlin une cardinalité 0..1 donne un type **nullable** : `X?`

Kotlin une cardinalité 0..* ou 1..* donne un type `Array<X?>`
(ou une autre collection)

Kotlin les visibilités UML doivent être **scrupuleusement** respectées

Kotlin On respecte **scrupuleusement** tous les noms donnés en UML

Aller plus loin (en Kotlin)

On reviendra plus tard sur

- des précisions sur les constructeurs
 - ▶ écriture simplifiée
 - ▶ constructeurs secondaires
- **Héritage**
 - ▶ classes abstraites et généralisation
 - ▶ interfaces et réalisation
- redéfinition de `toString()` de `equals(...)`
- redéfinition des opérateurs `+`, `==`, `>` ...